General Review of Hash-Based Signatures

Halil İbrahim Kaplan[1][0009−0006−0120−6512]

Cryptanalysis Laboratory, TÜBİTAK BİLGEM, Kocaeli, Turkey
halil.kaplan@tubitak.gov.tr

Abstract. Asymmetric cryptography addresses critical security needs such as authentication, data origin verification, and key distribution. However, the emergence of quantum computing threatens the foundations of most existing asymmetric schemes. Shor's algorithm allows polynomial time factorization on quantum computers, jeopardizing the hardness assumptions of RSA and ECC. In response, the NIST launched a standardization process for post-quantum cryptographic algorithms in 2016, stimulating research in alternative paradigms such as lattice-based, code-based, multivariate polynomial, isogeny-based, and hash-based schemes. Among them, hash-based signature algorithms stand out due to their conceptual simplicity and reliance on well-understood primitives like hash functions. This paper explores the structure and operation of hash-based signature schemes including XMSS, $XMSS^{MT}$, and SPHINCS+, and discusses known cryptanalytic techniques targeting them, such as intermediate secret-guessing and forgery attacks.

Keywords: Post-quantum cryptography · Hash-based signatures · XMSS · SPHINCS+ · Cryptanalysis

1 Basic Structures

In this section, the basic structures needed to understand hash-based signature algorithms will be explained.

1.1 Hash Functions

Cryptographic hash functions are functions that convert input data of any length into a fixed-length digest value. In order for these functions to be used securely, they must satisfy certain fundamental properties. These properties are as follows:

1. **Collision Resistance**: The probability that two different inputs produce the same hash value must be very low. In other words, the condition $h(x) = h(y)$ for $x \neq y$ should be extremely rare. This property is one of the cornerstones of hash function security, and finding collisions should be computationally infeasible.
2. **Pre-image Resistance**: It must be difficult to find the input that produces a given hash value. That is, given any hash output $h(x)$, it should be computationally hard to find the corresponding input x.

3. **Second Pre-image Resistance**: Given a specific input and its hash value, it should be difficult to find a different input that produces the same hash value. In other words, given $h(x)$, it should be computationally hard to find a different $y \neq x$ such that $h(y) = h(x)$.
4. **Avalanche Effect**: A small change in the input should result in a significant change in the output hash value. This property reduces the predictability of the hash function and ensures that even minor input changes create vastly different hash results.

1.2 Lamport One-Time Signature

The Lamport signature scheme is the first known hash-based signature algorithm. This scheme generates signatures that can only be used once, hence requiring a different key pair for each message. The steps of the scheme are as follows:

Lamport Key Generation Let us assume that a 256-bit message will be signed. In the Lamport algorithm, each bit of the signature requires 2 keys, resulting in a total of 512 public and private keys. The private keys are generated randomly and grouped as follows:

$$\mathrm{sk}_0 = \mathrm{sk}_{10}, \mathrm{sk}_{20}, \ldots, \mathrm{sk}_{2560}$$
$$\mathrm{sk}_1 = \mathrm{sk}_{11}, \mathrm{sk}_{21}, \ldots, \mathrm{sk}_{2561}$$

The public keys are obtained by applying a hash function to each private key:

$$\mathrm{pk}_0 = H(\mathrm{sk}_{10}), H(\mathrm{sk}_{20}), \ldots, H(\mathrm{sk}_{2560})$$
$$\mathrm{pk}_1 = H(\mathrm{sk}_{11}), H(\mathrm{sk}_{21}), \ldots, H(\mathrm{sk}_{2561})$$

Lamport Signature Each bit of the message is examined. If the bit is 0, the corresponding part of the signature is taken from the sk_0 group; if the bit is 1, it is taken from the sk_1 group.

For example, suppose the first three bits of our message are "001". Then, the first three parts of the signature would be:

$$\mathrm{sk}_{10} \| \mathrm{sk}_{20} \| \mathrm{sk}_{31}$$

Signature Verification The verifier examines each bit of the message one by one. If the bit value is 0, the hash of the corresponding part of the signature is computed and compared with the corresponding part of pk_0. If the bit is 1, it is compared with the corresponding part of pk_1.

This process is repeated for all bits. If all comparisons are equal, then the signature is successfully verified.

1.3 WOTS and WOTS+

The WOTS (Winternitz One-Time Signature) algorithm, like the Lamport signature algorithm, is a one-time signature scheme. While in the Lamport algorithm each bit of the message is signed individually, in the WOTS scheme the message is divided into chunks of w bits, and each chunk is signed instead of individual bits.

As a result, the signature size becomes w times smaller compared to the Lamport scheme. However, a w-bit message chunk can take 2^w different values, and a private key is required for each possible value.

This problem is also addressed using hash functions.

WOTS Key generation Let us assume that the message is divided into N parts, each consisting of l bits. In this case, N secret keys $\mathrm{sk}_0, \mathrm{sk}_1, \ldots, \mathrm{sk}_N$ are randomly generated. The 2^l possible values that each part can take are obtained by applying a hash function repeatedly to the randomly generated keys.

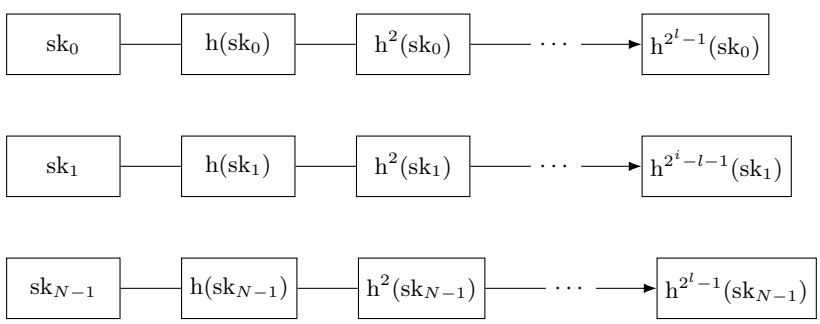

Fig. 1. WOTS key generation

In Figure 1, the secret keys are shown. The public key is obtained by applying the hash function 2^l times to each secret key (i.e., the 2^l-th hash).

For example, if the value of the first message chunk is 0, then sk_0 is used in the signature. If the value is $2^l - 1$, then the signature value is $h^{2^l-1}(\mathrm{sk}_0)$.

WOTS Signature For each part of the message, the appropriate number of hash iterations is applied to the corresponding secret key. These hashed values are concatenated to form the complete signature.

For example, suppose the first three chunks of the message are:

$$001 \| 010 \| 111$$

Then the first three parts of the signature would be:

$$h(\mathrm{sk}_0) \| h^2(\mathrm{sk}_1) \| h^7(\mathrm{sk}_2)$$

WOTS Signature Verification The verifier examines each message part individually. By checking the numerical value of each chunk, the verifier determines how many hash iterations are needed to reach the public key. It then hashes the corresponding part of the signature that many times. The result is compared with the known public key. If all values match, the signature is considered valid.

WOTS+ Fark yazılacak.

1.4 Merkle Trees and Multi-Signatures

Lamport and WOTS schemes are one-time signature algorithms, meaning they allow signing only a single message. However, in practical scenarios, signature algorithms are expected to support multiple signatures. To convert one-time signature schemes into algorithms that can sign multiple messages, **Merkle trees** are used.

As an example, the WOTS scheme can be used together with a Merkle tree. Figure 2 shows the sketch of merkle tree used with WOTS. The bottom leaves of the Merkle tree are the hash values of the WOTS public keys. Using a Merkle tree of height h, a signature scheme capable of signing 2^h messages can be constructed.

In addition to increasing signature capacity, Merkle trees also have the advantage of **reducing public key size**. Normally, a WOTS public key consists of multiple components—one for each message chunk—but when used with a Merkle tree, only **one public key** is required: the root of the Merkle tree.To verify a signature, the verifier must access certain intermediate nodes of the Merkle tree in order to compute the root. These intermediate values can be included as part of the signature, thus enabling successful verification.

1.5 L-tree and Hashing public key

Deneme When combining one-time signature schemes with a Merkle tree, the public key of the one-time signature scheme is selected as the bottom leaf of the Merkle tree. However, in the WOTS algorithm, a single signature includes not one but multiple public key components—one for each message part.

To resolve this issue, two different approaches are employed:

1. **L-tree (used in XMSS):** The L-tree is a specialized Merkle tree used to compress multiple public key components into a single public key. It repeatedly hashes pairs of public key elements until only one remains.
2. **Hashing (used in SPHINCS+):** In this method, all public key components are concatenated, and a single hash value is computed over the entire concatenation. This resulting hash serves as the single public key for Merkle tree integration.

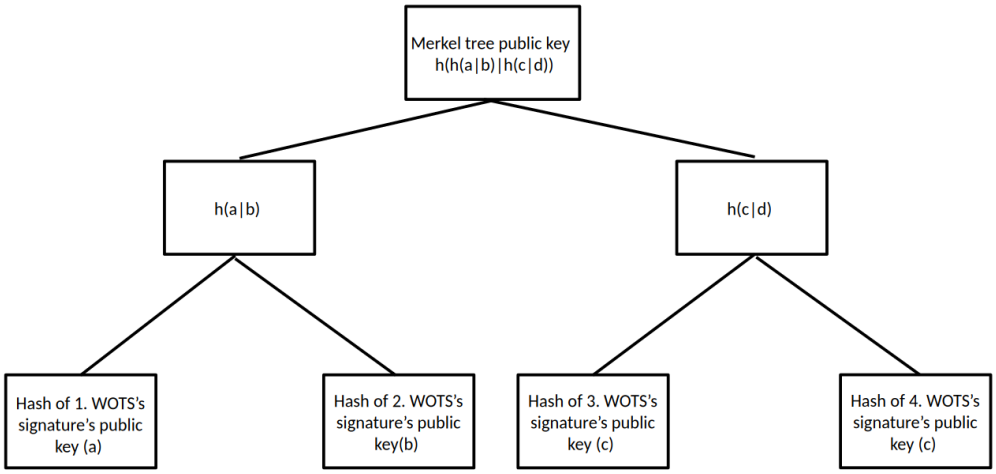

Fig. 2. Merkle Tree

1.6 FORS

The **FORS algorithm** (Forest of Random Subsets) is a *few-time signature scheme*. Unlike the WOTS algorithm, in FORS, when the message is divided into parts, a secret key is generated for each possible value of every part, and a small Merkle tree is constructed from these keys. The signature consists of:

- The secret key corresponding to the integer value of the message part.
- The necessary authentication path (intermediate nodes) to reach the root of the Merkle tree.

An example FORS signature for the below message is shown in Figure 3.

100 011 011 101 111 000

1.7 Address Structures in Hash-Based Signature Schemes

Hash-based signature schemes consist of either very large Merkle trees or multi-layered Merkle trees in order to increase their signature capacity. Since the same hash function is repeatedly used within the tree and similar operations are performed, **address structures** are used in hash-based signature schemes to:

- distinguish each operation,
- determine the position within the tree,
- track how many hash operations have been applied,
- and provide input differentiation for the hash functions.

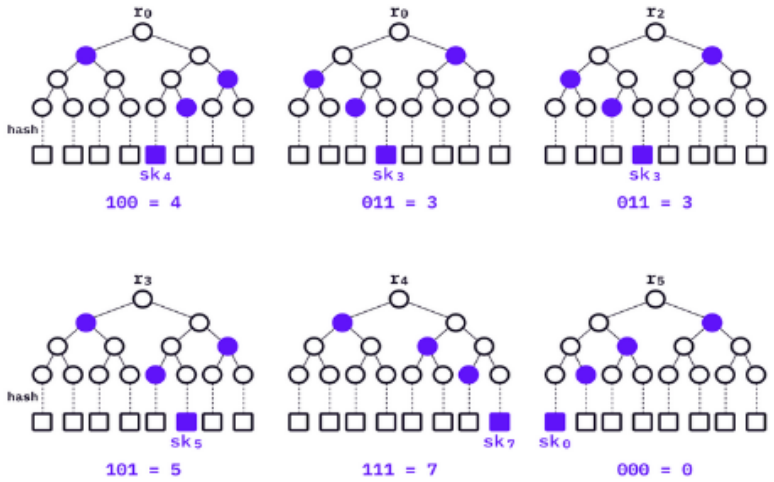

Fig. 3. FORS Signature example

Figure 4 shows the **OTS address structure** used in the XMSS algorithm [2]. In the example structure, the following fields appear in order from top to bottom:

- the layer in the multi-tree,
- the specific tree identifier,
- address type (e.g., OTS),
- index of the L-tree–WOTS pair,
- WOTS chain identifier,
- index of the hash operation within the chain,
- a flag indicating whether the hash value is used for encryption or masking.

Depending on the algorithm and its internal structure, the format of the address field can vary.

1.8 Stateful and Stateless Hash-Based Signature Schemes

Hash-based signature algorithms can be categorized into two types: **stateful** and **stateless**. The fundamental difference between these two categories lies in the management of state information during the signature generation and verification processes.

Stateful hash-based signature algorithms require the maintenance of state information while generating signatures. These algorithms rely on *one-time keys* that can be used only once per signature operation. Reusing these keys leads to serious security vulnerabilities. A representative example of such an algorithm is XMSS [2].

```
+-------------------------+
| layer address  (32 bits)|
+-------------------------+
| tree address   (64 bits)|
+-------------------------+
| type = 0       (32 bits)|
+-------------------------+
| OTS address    (32 bits)|
+-------------------------+
| chain address  (32 bits)|
+-------------------------+
| hash address   (32 bits)|
+-------------------------+
| keyAndMask     (32 bits)|
+-------------------------+
```

Fig. 4. XMSS OTS Address

Stateless hash-based signature algorithms, in contrast, do not require any state information to be stored during the signing process. A prominent example of a stateless algorithm is SPHINCS+ [1].

2 XMSS and $XMSS^{MT}$

References

[1] Daniel J Bernstein et al. "The SPHINCS+ signature framework". In: *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*. 2019, pp. 2129–2146.

[2] A Huelsing et al. *Rfc 8391: Xmss: extended merkle signature scheme*. 2018.